Errefaktorizazioa eclipsen laborategia

Yeray Ordoki

Ekaitz Rubio

Unai Redondo

Aurkibidea

1. Write short units of code Ekaitz Rubio 2

Arazoaren deskribapena 3

1. Hasierako kodea 3

2. Errefaktorizatutakoa 4

2. Write simple units of code Ekaitz Rubio 5

Arazoaren deskribapena 6

1. Hasierako kodea 6

2. Errefaktorizatutakoa 7

3. Duplicatecode Ekaitz Rubio 8

Arazoaren deskribapena 8

1. Hasierako kodea 8

2. Errefaktorizatutakoa 9

4.Keep unit interfaces small Ekaitz Rubio 10

Arazoaren deskribapena 11

1. Hasierako kodea 11

2. Errefaktorizatutakoa 12

1. errefaktorizazioa — Write short units of code (Yeray Ordoki) 13

Arazoaren deskribapena 13

1. Hasierako kodea 13

2. Errefaktorizatutako kodea 14

3. Egindako errefaktorizazioaren deskribapena 15

2. errefaktorizazioa — Write simple units of code (Yeray Ordoki) 15

1. Hasierako kodea 15

2. Errefaktorizatutako kodea 16

3. Egindako errefaktorizazioaren deskribapena 17

3. errefaktorizazioa — Duplicate code (Yeray Ordoki) 17

1. Hasierako kodea 17

2. Errefaktorizatutako kodea 17

3. Egindako errefaktorizazioaren deskribapena 17
4. errefaktorizazioa — Keep unit interfaces small (Yeray Ordoki) 17
1. Hasierako kodea 17
2. Errefaktorizatutako kodea 17
DataAccess klaseko metodo eguneratua 17
Sortutako SaleDetails klasea 17
3. Egindako errefaktorizazioaren deskribapena 17
1. Write short units of code (Unai Redondo) 17
Arazoaren deskribapena 17
1. Hasierako kodea 17
2. Errefaktorizatutakoa 17
2. Write simple units of code (Unai Redondo) 17
Arazoaren deskribapena 17
Hasierako kodea: 17
Errefaktorizatuta: 17
3. Duplicate code (Unai Redondo) 17
Arazoaren deskribapena 17
Hasierako kodea: 17
Errefaktorizatua: 17
4. Keep unitinterfaces small (Unai Redondo) 17
Arazoaren deskribapena 17
Hasierako kodea: 17
Errefaktorizatua: 17

Github: https://github.com/YerayOrdoki/refaktorizazioa

1. Write short units of code Ekaitz Rubio

Arazoaren deskribapena

DataAccess klaseko createEskaeraOferta() metodoak eskariaren eskaintzaren sorrera, bilaketa eta persistentszia prozesu osoa leku berean kudeatzen zuen, eta horrek metodoaren tamaina eta ardurak handitzen zituen (Write short units of code).

Mugako tamaina murrizteko eta kodearen mantengarritasuna hobetzeko, Extract Method eragiketa erabili da: entitateen bilaketa, egiaztapena eta eskaintzaren persistentszia processEskaeraOferta() izeneko metodo pribatu batera atera dira. Metodo nagusiak soilik transakzioaren fluxua (begin, commit, rollback) eta deia kudeatzen ditu.

1. Hasierako kodea

```
public boolean createEskaeraOferta(
            Integer eskID,
            String selEmail,
            float amount,
            String message) {

        if (eskID == null
                || selEmail == null
                || message == null
                || message.trim().isEmpty()) {
            return false;
        }

        try {
            db.getTransaction().begin();

            Eskaera eskaera = db.find(
                    Eskaera.class,
                    eskID);

            Seller seller = db.find(
                    Seller.class,
                    selEmail);

            if (eskaera == null || seller == null) {
                db.getTransaction().rollback();
                return false;
            }

            EskaeraOferta oferta = eskaera.addOferta(
                    amount,
```

```
                    message,
                    seller);

            db.persist(oferta);
            db.merge(eskaera);

            db.getTransaction().commit();

            return true;

        } catch (Exception e) {
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }

            return false;
        }
    }
```

2. Errefaktorizatutakoa

```
public boolean createEskaeraOferta(
            Integer eskID,
            String selEmail,
            float amount,
            String message) {

        if (eskID == null
                || selEmail == null
                || message == null
                || message.trim().isEmpty()) {
            return false;
        }

        try {
            db.getTransaction().begin();

            if (!processEskaeraOferta(eskID, selEmail, amount,
message)) {
                db.getTransaction().rollback();
                return false;
            }

            db.getTransaction().commit();

            return true;

        } catch (Exception e) {
```

```
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }

            return false;
        }
    }

    private boolean processEskaeraOferta(
            Integer eskID,
            String selEmail,
            float amount,
            String message) {

        Eskaera eskaera = db.find(
                Eskaera.class,
                eskID);

        Seller seller = db.find(
                Seller.class,
                selEmail);

        if (eskaera == null || seller == null) {
            return false;
        }

        EskaeraOferta oferta = eskaera.addOferta(
                amount,
                message,
                seller);

        db.persist(oferta);
        db.merge(eskaera);

        return true;
    }
```

2. Write simple units of code Ekaitz Rubio

Arazoaren deskribapena

DataAccess klaseko updateProfile() metodoak parametroen nulu-egiaztapenak, transakzioen kudeaketa, baldintzak eta bloke konmutatuak biltzen zituen, puntu-bakarren edo adarkatze-puntuen kopurua handituz (Write simple units of code jarraibidea hautsi nahiz).

Konplexutasuna eta adarkatze-puntuak murrizteko, Eclipseko Extract Method eragiketa erabili da:

1. Sarrerako datuen balidazioa hasInvalidProfileData() metodo pribatura atera da.
2. Saltzailearen datuen eguneraketa eta fusioa (merge) updateSellerData() metodo pribatura bideratu dira.

Modu honetan, metodo nagusiaren adarkatze-puntuak eta ardurak nabarmen gutxitzen dira, metodo sinpleagoak, irakurriagoak eta errazago testatzen direnak lortuz.

1. Hasierako kodea

```
public boolean updateProfile(String email, String newName, String
newPassword) {
        if (email == null || newName == null || newPassword ==
null) {
            return false;
        }

        try {
            db.getTransaction().begin();

            Seller seller = db.find(Seller.class, email);

            if (seller != null) {
                seller.setName(newName);
                seller.setPass(newPassword);

                db.merge(seller);
                db.getTransaction().commit();

                return true;
            }

            db.getTransaction().commit();

            return false;

        } catch (Exception e) {
```

```
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }

            return false;
        }
    }
```

2. Errefaktorizatutakoa

```
public boolean updateProfile(String email, String newName, String
newPassword) {
        if (hasInvalidProfileData(email, newName, newPassword)) {
            return false;
        }

        try {
            db.getTransaction().begin();

            Seller seller = db.find(Seller.class, email);

            if (seller != null) {
                updateSellerData(newName, newPassword, seller);
                db.getTransaction().commit();

                return true;
            }

            db.getTransaction().commit();

            return false;

        } catch (Exception e) {
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }

            return false;
        }
    }

private void updateSellerData(String newName, String newPassword,
Seller seller) {
seller.setName(newName);
seller.setPass(newPassword);

db.merge(seller);
}
```

```
private boolean hasInvalidProfileData(String email, String
newName, String newPassword) {
return email == null || newName == null || newPassword == null;
}
```

3. Duplicate code Ekaitz Rubio

Arazoaren deskribapena

DataAccess klaseko isLogged() eta getEmail() metodoek kredentzialak egiaztatzeko eta saltzailea datu-basean bilatzeko kontsulta-bloke berdina errepikatzen zuten (Duplicate code). Bi metodoek JPQL query bera sortzen zuten eta emaitzen zerrenda berdin aztertzen zuten.

Bikoizketa ezabatzeko eta kode berrerabilgarria idazteko, kontsultaren exekuzio eta bilaketa logika findSellerByCredentials() izeneko metodo pribatu batera atera da. Orain, bi metodo nagusiek metodo berri honi deitzen diote, eta kodearen mantengarritasuna hobetzen da, etorkizunean kontsulta edo autentikazio-logika aldatu beharrekoa bada leku bakarrean aldatu behar delako.

1. Hasierako kodea

```
public Emaitza isLogged(String log, String pass) {
        if (log == null || pass == null) {
            return new Emaitza(false, "", null, null);
        }

        TypedQuery<Seller> query = db.createQuery(
                "SELECT s FROM Seller s WHERE s.name=?1 AND
s.pass=?2",
                Seller.class);

        query.setParameter(1, log);
        query.setParameter(2, pass);

        if (!query.getResultList().isEmpty()) {
            Seller seller = query.getResultList().get(0);

            return new Emaitza(
                    true,
                    seller.getEmail(),
                    seller,
                    null);
        }
```

```
        return new Emaitza(false, "", null, null);
    }

    public String getEmail(String name, String pass) {
        if (name == null || pass == null) {
            return "";
        }

        TypedQuery<Seller> query = db.createQuery(
                "SELECT s FROM Seller s WHERE s.name=?1 AND
s.pass=?2",
                Seller.class);

        query.setParameter(1, name);
        query.setParameter(2, pass);

        if (!query.getResultList().isEmpty()) {
            return query.getResultList().get(0).getEmail();
        }

        return "";
    }
```

2. Errefaktorizatutakoa

```
public Emaitza isLogged(String log, String pass) {
        if (log == null || pass == null) {
            return new Emaitza(false, "", null, null);
        }

        Seller seller = findSellerByCredentials(log, pass);

        if (seller != null) {
            return new Emaitza(
                    true,
                    seller.getEmail(),
                    seller,
                    null);
        }

        return new Emaitza(false, "", null, null);
    }

    public String getEmail(String name, String pass) {
        if (name == null || pass == null) {
            return "";
        }

        Seller seller = findSellerByCredentials(name, pass);
```

```
        if (seller != null) {
            return seller.getEmail();
        }

        return "";
    }

    private Seller findSellerByCredentials(String name, String
pass) {
        TypedQuery<Seller> query = db.createQuery(
                "SELECT s FROM Seller s WHERE s.name=?1 AND
s.pass=?2",
                Seller.class);

        query.setParameter(1, name);
        query.setParameter(2, pass);

        List<Seller> results = query.getResultList();
        if (!results.isEmpty()) {
            return results.get(0);
        }

        return null;
    }
```

4.Keep unit interfaces small Ekaitz Rubio

Arazoaren deskribapena

DataAccess klaseko createReview() metodoak hainbat parametro jasotzen zituen (authorEmail, saleNumber, comment, stars). Interfazeak txikitzeko eta parametro kopurua mugarik gabe ez hazteko, Introduce Parameter Object eragiketa erabili da.

Balorazioaren edukia eta izar kopurua dataAccess paketean sortutako ReviewDetails klase berrian bildu dira. Klase honek bere atributu pribatuak, eraikitzailea eta dagozkion getter/setter metodoak ditu.

Errefaktorizazioaren ondoren, createReview() metodoak parametro gutxiago jasotzen ditu eta datuak objektu baten bidez pasatzen dira. Honek kodearen irakurgarritasuna eta etorkizuneko hedapenak askoz errazago kudeatzea ahalbidetzen du.

1. Hasierako kodea

```
public void createReview(
            String authorEmail,
            int saleNumber,
            String comment,
            int stars) {

        if (stars < 1 || stars > 5) {
            throw new RuntimeException(
                    "Las estrellas deben estar entre 1 y 5.");
        }

        db.getTransaction().begin();

        try {
            Sale sale = db.find(
                    Sale.class,
                    saleNumber);

            Seller author = db.find(
                    Seller.class,
                    authorEmail);

            Seller target = sale.getSeller();

            if (sale.getStatus() != 10) {
                throw new RuntimeException(
                        "Solo puedes valorar productos ya
comprados.");
            }
```

```
            Review newReview = new Review(
                    author,
                    target,
                    sale,
                    comment,
                    stars);

            target.addReview(newReview);

            db.persist(newReview);

            db.getTransaction().commit();

        } catch (Exception e) {
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }

            throw e;
        }
    }
```

2. Errefaktorizatutakoa

```
public void createReview(
            String authorEmail,
            int saleNumber,
            ReviewDetails reviewDetails) {

        if (reviewDetails.getStars() < 1 ||
reviewDetails.getStars() > 5) {
            throw new RuntimeException(
                    "Las estrellas deben estar entre 1 y 5.");
        }

        db.getTransaction().begin();

        try {
            Sale sale = db.find(
                    Sale.class,
                    saleNumber);

            Seller author = db.find(
                    Seller.class,
                    authorEmail);
```

```
            Seller target = sale.getSeller();

            if (sale.getStatus() != 10) {
                throw new RuntimeException(
                        "Solo puedes valorar productos ya
comprados.");
            }

            Review newReview = new Review(
                    author,
                    target,
                    sale,
                    reviewDetails.getComment(),
                    reviewDetails.getStars());

            target.addReview(newReview);

            db.persist(newReview);

            db.getTransaction().commit();

        } catch (Exception e) {
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }

            throw e;
        }
    }
```

1. errefaktorizazioa — Write short units of code (Yeray Ordoki)

Arazoaren deskribapena

`DataAccess` klaseko `buyBasket()` metodoak saski osoaren erosketa kudeatzen zuen, eta, aldi berean, produktu bakoitza erosteko instrukzio guztiak barne zituen. Horrek metodoaren luzera handitzen zuen eta bi ardura maila nahasten zituen: saskiaren prozesamendua eta produktu bakarraren erosketa.

Errefaktorizazioaren helburua metodo luzea metodo txikiagoetan banatzea da, bakoitzak gehienez 15 kode-lerro izan ditzan.

1. Hasierako kodea

```
java
public void buyBasket(String email) {
    db.getTransaction().begin();

    Seller buyer = db.find(Seller.class, email);
    List<Sale> basket = buyer.getBasket();

    for (Sale s : new ArrayList<Sale>(basket)) {
        Sale sale = db.find(Sale.class, s.getSaleNumber());
        Seller sellerOfProduct = sale.getSeller();

        buyer.setDirua(-sale.getPrice());
        sellerOfProduct.setDirua(sale.getPrice());

        sale.setStatus(10);

        buyer.addBought(sale);
    }

    buyer.clearBasket();

    db.getTransaction().commit();
}
```

2. Errefaktorizatutako kodea

```
public void buyBasket(String email) {
    db.getTransaction().begin();

    Seller buyer = db.find(Seller.class, email);
    List<Sale> basket = buyer.getBasket();

    for (Sale s : new ArrayList<Sale>(basket)) {
        purchaseBasketItem(buyer, s);
    }

    buyer.clearBasket();

    db.getTransaction().commit();
}
```

```
private void purchaseBasketItem(Seller buyer, Sale s) {
    Sale sale = db.find(Sale.class, s.getSaleNumber());
    Seller sellerOfProduct = sale.getSeller();

    buyer.setDirua(-sale.getPrice());
    sellerOfProduct.setDirua(sale.getPrice());

    sale.setStatus(10);

    buyer.addBought(sale);
}
```

3. Egindako errefaktorizazioaren deskribapena

Eclipseko `Refactor > Extract Method` eragiketa erabili da. `buyBasket()` metodoaren `for` begiztaren barruko instrukzioak hautatu dira, eta `purchaseBasketItem()` izeneko metodo pribatura atera dira.

Metodo berriak bi parametro jasotzen ditu:

- `buyer`: erosketa egiten duen saltzaile/erabiltzaile objektua.
- `s`: prozesatu beharreko saskiko produktua.

Jatorrizko blokean zegoen kodearen ordez, Eclipsek honako deia sortu du:

```
purchaseBasketItem(buyer, s);
```

2. errefaktorizazioa — Write simple units of code (Yeray Ordoki)

1. Hasierako kodea

```
public void buySale(Sale s, Seller current) {
    if (s == null || current == null
            || s.getSaleNumber() == null
            || current.getEmail() == null) {
        return;
    }
```

```
    try {
        db.getTransaction().begin();

        Sale foundS = db.find(Sale.class, s.getSaleNumber());
        Seller foundCurrent = db.find(Seller.class,
current.getEmail());

        if (foundS == null || foundCurrent == null) {
            throw new RuntimeException("Sale or buyer not found");
        }

        foundS.setStatus(10);

        float price = s.getPrice();

        foundCurrent.setDirua(-price);
        foundCurrent.addBought(foundS);

        Seller seller = foundS.getSeller();

        seller.setDirua(price);

        foudb.merge(seller);

        db.getTransaction().commit();

    } catch (Exception e) {
        if (db.getTransaction().isActive()) {
            db.getTransaction().rollback();
        }

        throw e;
    }
}
```

2. Errefaktorizatutako kodea

```
public void buySale(Sale s, Seller current) {
   if (hasInvalidPurchaseData(s, current)) {
        return;
    }

    try {
```

```
        db.getTransaction().begin();

        Sale foundS = db.find(Sale.class, s.getSaleNumber());
        Seller foundCurrent = db.find(Seller.class,
current.getEmail());

        if (isPurchaseEntityMissing(foundS, foundCurrent)) {
            throw new RuntimeException("Sale or buyer not found");
        }

        foundS.setStatus(10);

        float price = s.getPrice();

        foundCurrent.setDirua(-price);
        foundCurrent.addBought(foundS);

        Seller seller = foundS.getSeller();

        seller.setDirua(price);

        foundS = db.merge(foundS);
        foundCurrent = db.merge(foundCurrent);
        seller = db.merge(seller);

        db.getTransaction().commit();

    } catch (Exception e) {
        if (db.getTransaction().isActive()) {
            db.getTransaction().rollback();
        }

        throw e;
    }
}

private boolean isPurchaseEntityMissing(
        Sale foundS, Seller foundCurrent) {
    return foundS == null || foundCurrent == null;
}

private boolean hasInvalidPurchaseData(Sale s, Seller current) {
```

```
    return isPurchaseEntityMissing(s, current)
            || s.getSaleNumber() == null
            || current.getEmail() == null;
}
```

3. Egindako errefaktorizazioaren deskribapena

`DataAccess` klaseko `buySale()` metodoak baldintza konposatuak eta salbuespenen tratamendua zituen. `if`, `catch` eta `||` elementuak adarkatze-puntu gisa zenbatuta, metodoak 8 puntu zituen; hortaz, *Write simple units of code* jarraibideko 4 puntuko muga gainditzen zuen.

Konplexutasuna murrizteko, Eclipseko `Refactor > Extract Method` eragiketa bi aldiz erabili da:

> Sarrerako datuen balidazio-baldintza `hasInvalidPurchaseData()` metodo pribatura atera da.
> Salmenta edo eroslea `null` diren egiaztatzen duen baldintza `isPurchaseEntityMissing()` metodo pribatura atera da.

Bigarren erauzketan, Eclipsek baldintza baliokidea ordezkatu du `hasInvalidPurchaseData()` metodoaren barruan ere. Horrela, bi objektuen `null` egiaztapena metodo bakarrean definituta geratu da.

Azken kodearen adarkatze-puntuak honela banatzen dira:

Metodoa	Zenbaketaren oinarria	Adarkatze-puntuak
Hasierako `buySale()`	3 `if`, 4 `||` eta `catch` bat	8
Errefaktorizatutako `buySale()`	3 `if` eta `catch` bat	4
`hasInvalidPurchaseData()`	2 `||`	2
`isPurchaseEntityMissing()`	`||` bat	1

Beraz, errefaktorizazioan parte hartzen duten metodo guztiek gehienez 4 adarkatze-puntuko muga betetzen dute.

Jatorrizko egiaztapenak eta haien ebaluazio-ordena mantendu dira. `Sale` edo `Seller` objektua `null` denean, baldintza egiazkoa da eta ez dira ondorengo getter-ak ebaluatzen. Erosketaren instrukzioak, transakzioaren hasiera eta amaiera, rollback-a eta salbuespenaren birjaurtiketa ere aldatu gabe mantendu dira.

3. errefaktorizazioa — Duplicate code (Yeray Ordoki)

1. Hasierako kodea

```
public List<Sale> getSales(String desc) {
    System.out.println(">> DataAccess: getProducts=> from= " +
desc);

    List<Sale> res = new ArrayList<Sale>();

    TypedQuery<Sale> query = db.createQuery(
            "SELECT s FROM Sale s WHERE s.title LIKE ?1",
            Sale.class);

    query.setParameter(1, "%" + desc + "%");

    List<Sale> sales = query.getResultList();

    for (Sale sale : sales) {
        if (sale.getStatus() != 10) {
            res.add(sale);
        }
    }

    return res;
}

public List<Sale> getPublishedSales(String desc, Date pubDate) {
    System.out.println(">> DataAccess: getProducts=> from= " +
desc);

    List<Sale> res = new ArrayList<Sale>();

    TypedQuery<Sale> query = db.createQuery(
            "SELECT s FROM Sale s WHERE s.title LIKE ?1 AND
s.pubDate <= ?2",
            Sale.class);

    query.setParameter(1, "%" + desc + "%");
    query.setParameter(2, pubDate);

    List<Sale> sales = query.getResultList();
```

```
    for (Sale sale : sales) {
        if (sale.getStatus() != 10) {
            res.add(sale);
        }
    }

    return res;
}
```

2. Errefaktorizatutako kodea

```
public List<Sale> getSales(String desc) {
  System.out.println(">> DataAccess: getProducts=> from= " + desc);

    List<Sale> res = new ArrayList<Sale>();

    TypedQuery<Sale> query = db.createQuery(
            "SELECT s FROM Sale s WHERE s.title LIKE ?1",
            Sale.class);

    query.setParameter(1, "%" + desc + "%");

    List<Sale> sales = query.getResultList();

    filterUnsoldSales(res, sales);

    return res;
}

private void filterUnsoldSales(List<Sale> res, List<Sale> sales) {
    for (Sale sale : sales) {
        if (sale.getStatus() != 10) {
            res.add(sale);
        }
    }
}

public List<Sale> getPublishedSales(String desc, Date pubDate) {
    System.out.println(">> DataAccess: getProducts=> from= " +
desc);

    List<Sale> res = new ArrayList<Sale>();
```

```
    TypedQuery<Sale> query = db.createQuery(
            "SELECT s FROM Sale s WHERE s.title LIKE ?1 AND
s.pubDate <= ?2",
            Sale.class);

    query.setParameter(1, "%" + desc + "%");
    query.setParameter(2, pubDate);

    List<Sale> sales = query.getResultList();

    filterUnsoldSales(res, sales);

    return res;
}
```

3. Egindako errefaktorizazioaren deskribapena

`DataAccess` klaseko `getSales()` eta `getPublishedSales()` metodoetan salmentak iragazteko kode-bloke bera errepikatzen zen. Bi metodoek kontsultaren emaitzak zeharkatzen zituzten `for` begizta baten bidez, eta `sale.getStatus() != 10` baldintza betetzen zuten salmentak `res` zerrendara gehitzen zituzten.

Bikoizketa ezabatzeko, Eclipseko `Refactor > Extract Method` eragiketa erabili da. `getSales()` metodoan errepikatutako `for` blokea hautatu da, eta `filterUnsoldSales()` izeneko metodo pribatura atera da. Beste agerraldi baliokidea ere ordezkatu da, eta bi metodoek metodo berriari deitzen diote.

Sortutako metodoak bi parametro jasotzen ditu:

- `res`: iragazketa gainditzen duten salmentak jasotzen dituen emaitzen zerrenda.
- `sales`: kontsultatik lortutako eta zeharkatu beharreko salmenten zerrenda.

Metodoaren itzulera-mota `void` da, emaitzak parametro gisa jasotako `res` zerrendara gehitzen direlako. Jatorrizko metodoek zerrenda hori itzultzen jarraitzen dute.

Aldaketaren ondoren, iragazketa-logika leku bakarrean definituta geratu da. Iragazketa-baldintza etorkizunean aldatu behar bada, metodo bakarrean eguneratu ahal izango da, bi kopia bereizitan aldatu beharrean.

Jatorrizko portaera mantendu da:

- Metodo bakoitzaren kontsulta eta parametroak ez dira aldatu.
- `sale.getStatus() != 10` baldintza mantendu da.

Salmentak jatorrizko zerrendaren ordena berean gehitzen dira.
Metodo bakoitzak bere `res` zerrenda sortzen eta itzultzen jarraitzen du.

4. errefaktorizazioa — Keep unit interfaces small (Yeray Ordoki)

1. Hasierako kodea

```
public Sale createSale(
        String title,
       String description,
        int status,
        float price,
        Date pubDate,
        String sellerEmail,
        File file)
        throws FileNotUploadedException,
               MustBeLaterThanTodayException,
               SaleAlreadyExistException {

    System.out.println(">> DataAccess: createProduct=> title= "
            + title + " seller=" + sellerEmail);

    try {
        if (pubDate.before(UtilDate.trim(new Date()))) {
            throw new MustBeLaterThanTodayException(
                    ResourceBundle.getBundle(ETIQUETAS)

.getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
        }

        if (file == null) {
            throw new FileNotUploadedException(
                    ResourceBundle.getBundle(ETIQUETAS)

.getString("DataAccess.ErrorFileNotUploadedException"));
        }
```

```
        db.getTransaction().begin();

        Seller seller = db.find(Seller.class, sellerEmail);

        if (seller.doesSaleExist(title)) {
            db.getTransaction().commit();

            throw new SaleAlreadyExistException(
                    ResourceBundle.getBundle(ETIQUETAS)

.getString("DataAccess.SaleAlreadyExist"));
        }

        Sale sale = seller.addSale(
                title, description, status, price, pubDate, file);

        db.persist(sale);
        seller = db.merge(seller);

        db.getTransaction().commit();

        System.out.println("sale stored " + sale + " " + seller);

        return sale;

    } catch (NullPointerException e) {
        if (db.getTransaction().isActive()) {
            db.getTransaction().rollback();
        }

        return null;
    }
}
```

2. Errefaktorizatutako kodea

DataAccess klaseko metodo eguneratua

```
public Sale createSale(
       SaleDetails parameterObject,
        Date pubDate,
        String sellerEmail,
        File file)
        throws FileNotUploadedException,
```

```
                MustBeLaterThanTodayException,
                SaleAlreadyExistException {

    System.out.println(">> DataAccess: createProduct=> title= "
            + parameterObject.getTitle() + " seller=" +
sellerEmail);

    try {
        if (pubDate.before(UtilDate.trim(new Date()))) {
            throw new MustBeLaterThanTodayException(
                    ResourceBundle.getBundle(ETIQUETAS)

.getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
        }

        if (file == null) {
            throw new FileNotUploadedException(
                    ResourceBundle.getBundle(ETIQUETAS)

.getString("DataAccess.ErrorFileNotUploadedException"));
        }

        db.getTransaction().begin();

        Seller seller = db.find(Seller.class, sellerEmail);

        if (seller.doesSaleExist(parameterObject.getTitle())) {
            db.getTransaction().commit();

            throw new SaleAlreadyExistException(
                    ResourceBundle.getBundle(ETIQUETAS)

.getString("DataAccess.SaleAlreadyExist"));
        }

        Sale sale = seller.addSale(
                parameterObject.getTitle(),
                parameterObject.getDescription(),
                parameterObject.getStatus(),
                parameterObject.getPrice(),
                pubDate,
                file);
```

```
            db.persist(sale);
            seller = db.merge(seller);

            db.getTransaction().commit();

            System.out.println("sale stored " + sale + " " + seller);

            return sale;

        } catch (NullPointerException e) {
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }

            return null;
        }
}
```

Sortutako SaleDetails klasea

```
package dataAccess;

public class SaleDetails {
    private String title;
    private String description;
    private int status;
    private float price;

    public SaleDetails(
            String title,
            String description,
            int status,
            float price) {
        this.title = title;
        this.description = description;
        this.status = status;
        this.price = price;
    }

    public String getTitle() {
        return title;
    }
```

```
    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public int getStatus() {
        return status;
    }

    public void setStatus(int status) {
        this.status = status;
    }

    public float getPrice() {
        return price;
    }

    public void setPrice(float price) {
        this.price = price;
    }
}
```

3. Egindako errefaktorizazioaren deskribapena

`DataAccess` klaseko `createSale()` metodoak 7 parametro jasotzen zituen: `title`, `description`, `status`, `price`, `pubDate`, `sellerEmail` eta `file`. Parametro kopuru horrek *Keep unit interfaces small* jarraibidean ezarritako gehienez 4 parametroko muga gainditzen zuen.

Metodoaren interfazea txikitzeko, Eclipseko `Refactor > Introduce Parameter Object` eragiketa erabili da. Salmentaren oinarrizko lau datuak —`title`, `description`, `status` eta `price`— `dataAccess` paketeko `SaleDetails` klase berrian bildu dira.

Klase berriak lau atributu pribatu ditu, atributu horiek hasieratzeko eraikitzailea eta bakoitzaren getter eta setter metodoak.

Errefaktorizazioaren ondoren, `createSale()` metodoak honako 4 parametroak jasotzen ditu:

```
SaleDetails parameterObject
Date pubDate
String sellerEmail
File file
```

Metodoaren barruan, bildutako datuak getter-en bidez eskuratzen dira: `getTitle()`, `getDescription()`, `getStatus()` eta `getPrice()`. `SaleDetails` eraikitzaileak ere 4 parametro jasotzen ditu; beraz, parametro kopuruaren arazoa ez da zazpi parametroko beste eraikitzaile batera lekualdatu.

Aldaketan jatorrizko datu-erabilerak mantendu dira. Argitalpen-dataren eta fitxategiaren balidazioak, salmenta existitzen den egiaztapena, datu-baseko eragiketak eta transakzioaren kudeaketa lehen bezala geratu dira. Aldaketa nagusia lau datu bereizi objektu bakar baten bidez pasatzea izan da.

1. Write short units of code (Unai Redondo)

Arazoaren deskribapena

DataAccess klaseko createEskaera() metodoak 22 kode-lerro zituen. Transakzioaren hasiera/amaiera, saltzailearen bilaketa, eskaeraren sortzea, persistentszia eta merge eragiketak leku berean kudeatzen zituen. Horrek Write short units of code jarraibideko gehienezko 15 lerroko muga gainditzen zuen.

Prozesua: Eclipse-ko Refactor > Extract Method eragiketa erabili da. Eskaera sortu, persistitu eta saltzaileari lotzeko instrukzioak persistAndLinkEskaera() izeneko metodo pribatu batera atera dira.

Hobekuntza: Metodo nagusiaren luzera nabarmen murriztu da (15 lerrotik behera geratu da), eta ardurak hobeto banatu dira: metodo nagusiak transakzioaren fluxua kudeatzen du eta laguntzaileak entitateen arteko lotura eta gordetzea. Jatorrizko funtzionalitatea eta probak erabat mantentzen dira.

1. Hasierako kodea

```
public Eskaera createEskaera(String title, String description, String status, String sellerEmail) {
```

```
      try {
      db.getTransaction().begin();
      Seller seller = db.find(Seller.class, sellerEmail);
      if (seller == null) {
      db.getTransaction().commit();
      return null;
      }
      Eskaera eskaera = seller.addEskaera(title, description, status);
      db.persist(eskaera);
      db.merge(seller);
      db.getTransaction().commit();
      return eskaera;
} catch (NullPointerException e) {
      if (db.getTransaction().isActive()) {
      db.getTransaction().rollback();
   }
      return null;
       }
}
```

2. Errefaktorizatutakoa

```
public Eskaera createEskaera(String title, String description, String
status, String sellerEmail) {
try {
      db.getTransaction().begin();
      Seller seller = db.find(Seller.class, sellerEmail);
      if (seller == null) {
      db.getTransaction().commit();
                                          return null;
```

```
}

     Eskaera eskaera = persistAndLinkEskaera(title, description,
status, seller);
     db.getTransaction().commit();
     return eskaera;
     } catch (NullPointerException e) {
     if (db.getTransaction().isActive()) {
     db.getTransaction().rollback();
}
          return null;
}
}

private Eskaera persistAndLinkEskaera(String title, String
description, String   status, Seller seller) {
     Eskaera eskaera = seller.addEskaera(title, description, status);
     db.persist(eskaera);
     db.merge(seller);
     return eskaera;
}
```

2. Write simple units of code (Unai Redondo)

Arazoaren deskribapena

isRegistered() metodoak sarrerako balio nuloak egiaztatzeko bi || eragile dituen baldintza konposatu bat zeukan, baita entitateen eta kontsulten ondorengo if adarkatzeak ere. Adarkatze-puntu guztiak zenbatuta (|| eta if), metodoaren

konplexutasun ziklomatikoa 6 zen, Write simple units of code jarraibideko gehienezko 4ko muga gaindituz.

Prozesua: Eclipse-ko Refactor > Extract Method eragiketa erabili da sarrerako parametro nuloak egiaztatzen dituen baldintza konposatua hasNullRegistrationParams() metodo pribatura ateratzeko.

Hobekuntza: Metodo nagusiaren konplexutasun ziklomatikoa murriztu da, adarkatze-puntuak laguntzaile-metodoaren eta metodo nagusiaren artean banatuz. Kodearen irakurgarritasuna eta probagarritasuna hobetu dira, jatorrizko balidazio guztiak mantenduz.

Hasierako kodea:

```
public Emaitza isRegistered(String mail, String user, String password)
{
if (mail == null || user == null || password == null) {
return new Emaitza(false, "", null, null);
}
      Seller seller = db.find(Seller.class, mail);
      Emaitza emaitza = new Emaitza(false, "", null, null);
      if (seller != null) {
      return emaitza;
}
TypedQuery<Seller> query = db.createQuery("SELECT s FROM Seller s
WHERE s.name=?1", Seller.class);
query.setParameter(1, user);
if (!query.getResultList().isEmpty()) {
return emaitza;
}
      db.getTransaction().begin();

      Seller sellerToPersist = new Seller(mail, user, password);
      emaitza = new Emaitza(true, mail, sellerToPersist, null);
      db.persist(sellerToPersist);
```

```
      db.getTransaction().commit();


   return emaitza;
}
```

Errefaktorizatuta:

```
public Emaitza isRegistered(String mail, String user, String password)
{
if (hasNullRegistrationParams(mail, user, password)) {
return new Emaitza(false, "", null, null);
}
Seller seller = db.find(Seller.class, mail);
Emaitza emaitza = new Emaitza(false, "", null, null);
if (seller != null) {
return emaitza;
}
TypedQuery<Seller> query = db.createQuery("SELECT s FROM Seller s
WHERE s.name=?1", Seller.class);
query.setParameter(1, user);
if (!query.getResultList().isEmpty()) {
return emaitza;
}
db.getTransaction().begin();
Seller sellerToPersist = new Seller(mail, user, password);
           emaitza = new Emaitza(true, mail, sellerToPersist, null);
db.persist(sellerToPersist);
           db.getTransaction().commit();
     return emaitza;
}
private boolean hasNullRegistrationParams(String mail, String user,
String password) {
```

```
return mail == null || user == null || password == null;
}
```

3. Duplicate code (Unai Redondo)

Arazoaren deskribapena

getBoughtsBySeller() eta getSalesBySeller() metodoetan (baita DataAccess klaseko beste metodo batzuetan ere) posta elektroniko bidez Seller entitatea datu-basean bilatzeko kode-blokea errepikatuta zegoen. Bikoizketa horrek Duplicate code ("Do not copy code") printzipioa urratzen zuen.

Prozesua: Eclipse-ko Refactor > Extract Method eragiketa erabili da findSeller() metodo pribatua sortzeko. Bikoiztutako bilaketa-blokeak metodo berriaren deiarekin ordezkatu dira.

Hobekuntza: Kode bikoiztua ezabatu da, Seller entitatearen bilaketa-logika leku bakar batean zentralizatuz. Etorkizunean bilaketa-logika edo nuloen kudeaketa aldatu behar bada, metodo bakar batean eguneratu ahal izango da. Jatorrizko portaera eta itzulera-motak erabat berdinak izaten jarraitzen dute.

Hasierako kodea:

```
public List<Sale> getBoughtsBySeller(String email) {
	Seller seller = db.find(Seller.class, email);
	if (seller == null) {
	return new ArrayList<Sale>();
}
	return seller.getBoughts();
}
	public List<Sale> getSalesBySeller(String email) {
	Seller seller = db.find(Seller.class, email);
	if (seller == null) {
	return new ArrayList<Sale>();
}
	return seller.getSales();
```

```
}
```

Errefaktorizatua:

```
 public List<Sale> getBoughtsBySeller(String email) {
Seller seller = findSeller(email);
     if (seller == null) {
     return new ArrayList<Sale>();
}
     return seller.getBoughts();
}
private Seller findSeller(String email) {
     return db.find(Seller.class, email);
}
public List<Sale> getSalesBySeller(String email) {
     Seller seller = findSeller(email);
     if (seller == null) {
     return new ArrayList<Sale>();
}
     return seller.getSales();

}
```

4. Keep unit interfaces small (Unai Redondo)

Arazoaren deskribapena

Jatorrizko createEskaeraOferta() metodoak 4 parametro bereizi jasotzen zituen (eskID, selEmail, amount, message), Keep unit interfaces small gidalerroaren mugan zegoelarik. Metodoen interfazeak ahalik eta txikienak izateko, gomendagarria da parametroak objektu bakar batean biltzea.

Prozesua: OfertaData izeneko Parameter Object klase berri bat sortu da dataAccess paketean. Eclipse-ko Refactor > Introduce Parameter Object erabiliz, lau parametroak OfertaData ofertaData objektu bakarrera murriztu dira.

Hobekuntza: Metodoaren interfazea nabarmen sinplifikatu da (4 parametro jasotzetik objektu bakarra jasotzera pasaz). Eskaintza baten datu guztiak enkapsulatuta geratzen dira, kodearen mantengarritasuna hobetuz eta interfaze garbia lortuz.

Hasierako kodea:

```
public boolean createEskaeraOferta(Integer eskID, String selEmail,
float amount, String message) {
    if (eskID == null
            || selEmail == null
            || message == null
            || message.trim().isEmpty()) {
        return false;
    }

    try {
        db.getTransaction().begin();

        if (!processEskaeraOferta(eskID, selEmail, amount, message))
{
            db.getTransaction().rollback();
            return false;
        }

        db.getTransaction().commit();

        return true;

    } catch (Exception e) {
```

```
        if (db.getTransaction().isActive()) {
            db.getTransaction().rollback();
        }

        return false;
    }
}

private boolean processEskaeraOferta(Integer eskID, String selEmail,
float amount, String message) {
    Eskaera eskaera = db.find(Eskaera.class, eskID);
    Seller seller = db.find(Seller.class, selEmail);

    if (eskaera == null || seller == null) {
        return false;
    }

    EskaeraOferta oferta = eskaera.addOferta(amount, message,
seller);

    db.persist(oferta);
    db.merge(eskaera);

    return true;
}
```

Errefaktorizatua:

```
public boolean createEskaeraOferta(OfertaData ofertaData) {
    if (ofertaData == null
            || ofertaData.getEskID() == null
```

```
            || ofertaData.getSelEmail() == null
            || ofertaData.getMessage() == null
            || ofertaData.getMessage().trim().isEmpty()) {
        return false;
    }
    try {
        db.getTransaction().begin();

        if (!processEskaeraOferta(ofertaData)) {
            db.getTransaction().rollback();
            return false;
        }
        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        if (db.getTransaction().isActive()) {
            db.getTransaction().rollback();
        }
        return false;
    }
}
private boolean processEskaeraOferta(OfertaData ofertaData) {
    Eskaera eskaera = db.find(Eskaera.class, ofertaData.getEskID());
    Seller seller = db.find(Seller.class, ofertaData.getSelEmail());
    if (eskaera == null || seller == null) {
        return false;
    }
    EskaeraOferta oferta = eskaera.addOferta(
            ofertaData.getAmount(),
```

```
            ofertaData.getMessage(),
            seller);
    db.persist(oferta);
    db.merge(eskaera);

    return true;
}
//Klase berria
package dataAccess;
public class OfertaData {
    private Integer eskID;
    private String selEmail;
    private float amount;
    private String message;

    public OfertaData(Integer eskID, String selEmail, float amount,
String message) {
        this.eskID = eskID;
        this.selEmail = selEmail;
        this.amount = amount;
        this.message = message;
    }
    public Integer getEskID() { return eskID; }
    public String getSelEmail() { return selEmail; }
    public float getAmount() { return amount; }
    public String getMessage() { return message; }
}
```